

# Reverse engineering

---

**Reverse engineering** (also known as **backwards engineering** or **back engineering**) is a process or method through which one attempts to understand through deductive reasoning how a previously made device, process, system, or piece of software accomplishes a task with very little (if any) insight into exactly how it does so. Depending on the system under consideration and the technologies employed, the knowledge gained during reverse engineering can help with repurposing obsolete objects, doing security analysis, or learning how something works.<sup>[1][2][3]</sup>



The Tupolev Tu-4, a Soviet bomber built by reverse engineering interned Boeing B-29 Superfortresses

Although the process is specific to the object on which it is being performed, all reverse engineering processes consist of three basic steps: information extraction, modeling, and review. Information extraction is the practice of gathering all relevant information for performing the operation. Modeling is the practice of combining the gathered information into an abstract model, which can be used as a guide for designing the new object or system. Review is the testing of the model to ensure the validity of the chosen abstract.<sup>[4]</sup> Reverse engineering is applicable in the fields of computer engineering, mechanical engineering, design, electrical and electronic engineering, civil engineering, nuclear engineering, aerospace engineering, software engineering, chemical engineering,<sup>[5]</sup> systems biology<sup>[6]</sup> and more.

## Overview

---

There are many reasons for performing reverse engineering in various fields. Reverse engineering has its origins in the analysis of hardware for commercial or military advantage.<sup>[7]:13</sup> However, the reverse engineering process may not always be concerned with creating a copy or changing the artifact in some way. It may be used as part of an analysis to deduce design features from products with little or no additional knowledge about the procedures involved in their original production.<sup>[7]:15</sup>

In some cases, the goal of the reverse engineering process can simply be a redocumentation of legacy systems.<sup>[7]:15[8]</sup> Even when the reverse-engineered product is that of a competitor, the goal may not be to copy it but to perform competitor analysis.<sup>[9]</sup> Reverse engineering may also be used to create interoperable products and despite some narrowly tailored United States and European Union legislation, the legality of using specific reverse engineering techniques for that purpose has been hotly contested in courts worldwide for more than two decades.<sup>[10]</sup>

Software reverse engineering can help to improve the understanding of the underlying source code for the maintenance and improvement of the software. Relevant information can be extracted to make a decision for software development and graphical representations of the code can provide alternate views regarding

the source code, which can help to detect and fix a software bug or vulnerability. Frequently, as some software develops, its design information and improvements are often lost over time, but that lost information can usually be recovered with reverse engineering. The process can also help to cut down the time required to understand the source code, thus reducing the overall cost of the software development.<sup>[11]</sup> Reverse engineering can also help to detect and to eliminate a malicious code written to the software with better code detectors. Reversing a source code can be used to find alternate uses of the source code, such as detecting the unauthorized replication of the source code where it was not intended to be used, or revealing how a competitor's product was built.<sup>[12]</sup> That process is commonly used for "cracking" software and media to remove their copy protection,<sup>[12]:7</sup> or to create a possibly improved copy or even a knockoff, which is usually the goal of a competitor or a hacker.<sup>[12]:8</sup>

Malware developers often use reverse engineering techniques to find vulnerabilities in an operating system to build a computer virus that can exploit the system vulnerabilities.<sup>[12]:5</sup> Reverse engineering is also being used in cryptanalysis to find vulnerabilities in substitution cipher, symmetric-key algorithm or public-key cryptography.<sup>[12]:6</sup>

There are other uses to reverse engineering:

- **Games.** Reverse engineering in the context of games and game engines is often used to understand underlying mechanics, data structures, and proprietary protocols, allowing developers to create mods, custom tools, or to enhance compatibility. This practice is particularly useful when interfacing with existing systems to improve interoperability between different game components, engines, or platforms. Platforms like ResHax<sup>[13]</sup> provide tools and resources that assist in analyzing game binaries, dissecting game engine behavior, thus contributing to a deeper understanding of game technology and enabling community-driven enhancements.
- **Interfacing.** Reverse engineering can be used when a system is required to interface to another system and how both systems would negotiate is to be established. Such requirements typically exist for interoperability.
- **Military or commercial espionage.** Learning about an enemy's or competitor's latest research by stealing or capturing a prototype and dismantling it may result in the development of a similar product or a better countermeasure against it.
- **Obsolescence.** Integrated circuits are often designed on proprietary systems and built on production lines, which become obsolete in only a few years. When systems using those parts can no longer be maintained since the parts are no longer made, the only way to incorporate the functionality into new technology is to reverse-engineer the existing chip and then to redesign it using newer tools by using the understanding gained as a guide. Another obsolescence originated problem that can be solved by reverse engineering is the need to support (maintenance and supply for continuous operation) existing legacy devices that are no longer supported by their original equipment manufacturer. The problem is particularly critical in military operations.
- **Product security analysis.** That examines how a product works by determining the specifications of its components and estimate costs and identifies potential patent infringement. Also part of product security analysis is acquiring sensitive data by disassembling and analyzing the design of a system component.<sup>[14]</sup> Another intent may be to remove copy protection or to circumvent access restrictions.
- **Competitive technical intelligence.** That is to understand what one's competitor is actually doing, rather than what it says that it is doing.
- **Saving money.** Finding out what a piece of electronics can do may spare a user from purchasing a separate product.
- **Repurposing.** Obsolete objects are then reused in a different-but-useful manner.

- **Design.** Production and design companies applied Reverse Engineering to practical craft-based manufacturing process. The companies can work on "historical" manufacturing collections through 3D scanning, 3D re-modeling and re-design. In 2013 Italian manufactures Baldi and Savio Firmino together with University of Florence optimized their innovation, design, and production processes.<sup>[15]</sup>

## Common uses

---

### Machines

As computer-aided design (CAD) has become more popular, reverse engineering has become a viable method to create a 3D virtual model of an existing physical part for use in 3D CAD, CAM, CAE, or other software.<sup>[16]</sup> The reverse-engineering process involves measuring an object and then reconstructing it as a 3D model. The physical object can be measured using 3D scanning technologies like CMMs, laser scanners, structured light digitizers, or industrial CT scanning (computed tomography). The measured data alone, usually represented as a point cloud, lacks topological information and design intent. The former may be recovered by converting the point cloud to a triangular-faced mesh. Reverse engineering aims to go beyond producing such a mesh and to recover the design intent in terms of simple analytical surfaces where appropriate (planes, cylinders, etc.) as well as possibly NURBS surfaces to produce a boundary-representation CAD model. Recovery of such a model allows a design to be modified to meet new requirements, a manufacturing plan to be generated, etc.

Hybrid modeling is a commonly used term when NURBS and parametric modeling are implemented together. Using a combination of geometric and freeform surfaces can provide a powerful method of 3D modeling. Areas of freeform data can be combined with exact geometric surfaces to create a hybrid model. A typical example of this would be the reverse engineering of a cylinder head, which includes freeform cast features, such as water jackets and high-tolerance machined areas.<sup>[17]</sup>

Reverse engineering is also used by businesses to bring existing physical geometry into digital product development environments, to make a digital 3D record of their own products, or to assess competitors' products. It is used to analyze how a product works, what it does, what components it has; estimate costs; identify potential patent infringement; etc.

Value engineering, a related activity that is also used by businesses, involves deconstructing and analyzing products. However, the objective is to find opportunities for cost-cutting.

### Printed circuit boards

Reverse engineering of printed circuit boards involves recreating fabrication data for a particular circuit board. This is done primarily to identify a design, and learn the functional and structural characteristics of a design. It also allows for the discovery of the design principles behind a product, especially if this design information is not easily available.

Outdated PCBs are often subject to reverse engineering, especially when they perform highly critical functions such as powering machinery, or other electronic components. Reverse engineering these old parts can allow the reconstruction of the PCB if it performs some crucial task, as well as finding alternatives which provide the same function, or in upgrading the old PCB.<sup>[18]</sup>

Reverse engineering PCBs largely follow the same series of steps. First, images are created by drawing, scanning, or taking photographs of the PCB. Then, these images are ported to suitable reverse engineering software in order to create a rudimentary design for the new PCB. The quality of these images that is necessary for suitable reverse engineering is proportional to the complexity of the PCB itself. More complicated PCBs require well lighted photos on dark backgrounds, while fairly simple PCBs can be recreated simply with just basic dimensioning. Each layer of the PCB is carefully recreated in the software with the intent of producing a final design as close to the initial. Then, the schematics for the circuit are finally generated using an appropriate tool.<sup>[19]</sup>

## Software

In 1990, the Institute of Electrical and Electronics Engineers (IEEE) defined (software) reverse engineering (SRE) as "the process of analyzing a subject system to identify the system's components and their interrelationships and to create representations of the system in another form or at a higher level of abstraction" in which the "subject system" is the end product of software development. Reverse engineering is a process of examination only, and the software system under consideration is not modified, which would otherwise be re-engineering or restructuring. Reverse engineering can be performed from any stage of the product cycle, not necessarily from the functional end product.<sup>[11]</sup>

There are two components in reverse engineering: redocumentation and design recovery. Redocumentation is the creation of new representation of the computer code so that it is easier to understand. Meanwhile, design recovery is the use of deduction or reasoning from general knowledge or personal experience of the product to understand the product's functionality fully.<sup>[11]</sup> It can also be seen as "going backwards through the development cycle".<sup>[20]</sup> In this model, the output of the implementation phase (in source code form) is reverse-engineered back to the analysis phase, in an inversion of the traditional waterfall model. Another term for this technique is program comprehension.<sup>[8]</sup> The Working Conference on Reverse Engineering (WCRE) has been held yearly to explore and expand the techniques of reverse engineering.<sup>[12][21]</sup> Computer-aided software engineering (CASE) and automated code generation have contributed greatly in the field of reverse engineering.<sup>[12]</sup>

Software anti-tamper technology like obfuscation is used to deter both reverse engineering and re-engineering of proprietary software and software-powered systems. In practice, two main types of reverse engineering emerge. In the first case, source code is already available for the software, but higher-level aspects of the program, which are perhaps poorly documented or documented but no longer valid, are discovered. In the second case, there is no source code available for the software, and any efforts towards discovering one possible source code for the software are regarded as reverse engineering. The second usage of the term is more familiar to most people. Reverse engineering of software can make use of the clean room design technique to avoid copyright infringement.

On a related note, black box testing in software engineering has a lot in common with reverse engineering. The tester usually has the API but has the goals to find bugs and undocumented features by bashing the product from outside.<sup>[22]</sup>

Other purposes of reverse engineering include security auditing, removal of copy protection ("cracking"), circumvention of access restrictions often present in consumer electronics, customization of embedded systems (such as engine management systems), in-house repairs or retrofits, enabling of additional features on low-cost "crippled" hardware (such as some graphics card chip-sets), or even mere satisfaction of curiosity.

## Binary software

Binary reverse engineering is performed if source code for a software is unavailable.<sup>[12]</sup> This process is sometimes termed *reverse code engineering*, or RCE.<sup>[23]</sup> For example, decompilation of binaries for the Java platform can be accomplished by using Jad. One famous case of reverse engineering was the first non-IBM implementation of the PC BIOS, which launched the historic IBM PC compatible industry that has been the overwhelmingly-dominant computer hardware platform for many years. Reverse engineering of software is protected in the US by the fair use exception in copyright law.<sup>[24]</sup> The Samba software, which allows systems that do not run Microsoft Windows systems to share files with systems that run it, is a classic example of software reverse engineering<sup>[25]</sup> since the Samba project had to reverse-engineer unpublished information about how Windows file sharing worked so that non-Windows computers could emulate it. The Wine project does the same thing for the Windows API, and OpenOffice.org is one party doing that for the Microsoft Office file formats. The ReactOS project is even more ambitious in its goals by striving to provide binary (ABI and API) compatibility with the current Windows operating systems of the NT branch, which allows software and drivers written for Windows to run on a clean-room reverse-engineered free software (GPL) counterpart.

## Binary software techniques

Reverse engineering of software can be accomplished by various methods. The three main groups of software reverse engineering are

1. Analysis through observation of information exchange, most prevalent in protocol reverse engineering, which involves using bus analyzers and packet sniffers, such as for accessing a computer bus or computer network connection and revealing the traffic data thereon. Bus or network behavior can then be analyzed to produce a standalone implementation that mimics that behavior. That is especially useful for reverse engineering device drivers. Sometimes, reverse engineering on embedded systems is greatly assisted by tools deliberately introduced by the manufacturer, such as JTAG ports or other debugging means. In Microsoft Windows, low-level debuggers such as SoftICE are popular.
2. Disassembly using a disassembler, meaning the raw machine language of the program is read and understood in its own terms, only with the aid of machine-language mnemonics. It works on any computer program but can take quite some time, especially for those who are not used to machine code. The Interactive Disassembler is a particularly popular tool.
3. Decompilation using a decompiler, a process that tries, with varying results, to recreate the source code in some high-level language for a program only available in machine code or bytecode.

## Software classification

Software classification is the process of identifying similarities between different software binaries (such as two different versions of the same binary) used to detect code relations between software samples. The task was traditionally done manually for several reasons (such as patch analysis for vulnerability detection and copyright infringement), but it can now be done somewhat automatically for large numbers of samples.

This method is being used mostly for long and thorough reverse engineering tasks (complete analysis of a complex algorithm or big piece of software). In general, statistical classification is considered to be a hard problem, which is also true for software classification, and so few solutions/tools that handle this task well.

## Source code

A number of UML tools refer to the process of importing and analysing source code to generate UML diagrams as "reverse engineering" .

Although UML is one approach in providing "reverse engineering" more recent advances in international standards activities have resulted in the development of the Knowledge Discovery Metamodel (KDM). The standard delivers an ontology for the intermediate (or abstracted) representation of programming language constructs and their interrelationships. An Object Management Group standard (on its way to becoming an ISO standard as well), KDM has started to take hold in industry with the development of tools and analysis environments that can deliver the extraction and analysis of source, binary, and byte code. For source code analysis, KDM's granular standards' architecture enables the extraction of software system flows (data, control, and call maps), architectures, and business layer knowledge (rules, terms, and process). The standard enables the use of a common data format (XMI) enabling the correlation of the various layers of system knowledge for either detailed analysis (such as root cause, impact) or derived analysis (such as business process extraction). Although efforts to represent language constructs can be never-ending because of the number of languages, the continuous evolution of software languages, and the development of new languages, the standard does allow for the use of extensions to support the broad language set as well as evolution. KDM is compatible with UML, BPMN, RDF, and other standards enabling migration into other environments and thus leverage system knowledge for efforts such as software system transformation and enterprise business layer analysis.

## Protocols

Protocols are sets of rules that describe message formats and how messages are exchanged: the protocol state machine. Accordingly, the problem of protocol reverse-engineering can be partitioned into two subproblems: message format and state-machine reverse-engineering.

The message formats have traditionally been reverse-engineered by a tedious manual process, which involved analysis of how protocol implementations process messages, but recent research proposed a number of automatic solutions.<sup>[26][27][28]</sup> Typically, the automatic approaches group observe messages into clusters by using various clustering analyses, or they emulate the protocol implementation tracing the message processing.

There has been less work on reverse-engineering of state-machines of protocols. In general, the protocol state-machines can be learned either through a process of offline learning, which passively observes communication and attempts to build the most general state-machine accepting all observed sequences of messages, and online learning, which allows interactive generation of probing sequences of messages and listening to responses to those probing sequences. In general, offline learning of small state-machines is known to be NP-complete,<sup>[29]</sup> but online learning can be done in polynomial time.<sup>[30]</sup> An automatic offline approach has been demonstrated by Comparetti et al.<sup>[28]</sup> and an online approach by Cho et al.<sup>[31]</sup>

Other components of typical protocols, like encryption and hash functions, can be reverse-engineered automatically as well. Typically, the automatic approaches trace the execution of protocol implementations and try to detect buffers in memory holding unencrypted packets.<sup>[32]</sup>

## Integrated circuits/smart cards

Reverse engineering is an invasive and destructive form of analyzing a smart card. The attacker uses chemicals to etch away layer after layer of the smart card and takes pictures with a scanning electron microscope (SEM). That technique can reveal the complete hardware and software part of the smart card. The major problem for the attacker is to bring everything into the right order to find out how everything works. The makers of the card try to hide keys and operations by mixing up memory positions, such as by bus scrambling.<sup>[33][34]</sup>

In some cases, it is even possible to attach a probe to measure voltages while the smart card is still operational. The makers of the card employ sensors to detect and prevent that attack.<sup>[35]</sup> That attack is not very common because it requires both a large investment in effort and special equipment that is generally available only to large chip manufacturers. Furthermore, the payoff from this attack is low since other security techniques are often used such as shadow accounts. It is still uncertain whether attacks against chip-and-PIN cards to replicate encryption data and then to crack PINs would provide a cost-effective attack on multifactor authentication.

Full reverse engineering proceeds in several major steps.

The first step after images have been taken with a SEM is stitching the images together, which is necessary because each layer cannot be captured by a single shot. A SEM needs to sweep across the area of the circuit and take several hundred images to cover the entire layer. Image stitching takes as input several hundred pictures and outputs a single properly overlapped picture of the complete layer.

Next, the stitched layers need to be aligned because the sample, after etching, cannot be put into the exact same position relative to the SEM each time. Therefore, the stitched versions will not overlap in the correct fashion, as on the real circuit. Usually, three corresponding points are selected, and a transformation applied on the basis of that.

To extract the circuit structure, the aligned, stitched images need to be segmented, which highlights the important circuitry and separates it from the uninteresting background and insulating materials.

Finally, the wires can be traced from one layer to the next, and the netlist of the circuit, which contains all of the circuit's information, can be reconstructed.

## Military applications

Reverse engineering is often used by people to copy other nations' technologies, devices, or information that have been obtained by regular troops in the fields or by intelligence operations. It was often used during the Second World War and the Cold War. Following are some well known examples from the Second World War and afterward:

- Jerry can: British and American forces in WW2 noticed that the Germans had gasoline cans with an excellent design. They reverse-engineered copies of those cans, which were popularly known as "Jerry cans".
- Nakajima G5N: In 1939, the U.S. Douglas Aircraft Company sold its DC-4E airliner prototype to Imperial Japanese Airways, which was secretly acting as a front for the Imperial Japanese Navy, which wanted a long-range strategic bomber but had been hindered by the Japanese aircraft industry's inexperience with heavy long-range aircraft. The DC-4E was transferred to the Nakajima Aircraft Company and dismantled for study; as a cover story, the

Japanese press reported that it had crashed in Tokyo Bay.<sup>[36][37]</sup> The wings, engines, and landing gear of the G5N were copied directly from the DC-4E.<sup>[38]</sup>

- Panzerschreck: The Germans captured an American bazooka during the Second World War and reverse engineered it to create the larger Panzerschreck.
- Tupolev Tu-4: In 1944, three American B-29 bombers on missions over Japan were forced to land in the Soviet Union. The Soviets, who did not have a similar strategic bomber, decided to copy the B-29. Within three years, they had developed the Tu-4, a nearly-perfect copy.<sup>[39]</sup>
- SCR-584 radar: copied by the Soviet Union after the Second World War, it is known for a few modifications - СЦР-584, Бинокль-Д.
- V-2 rocket: Technical documents for the V-2 and related technologies were captured by the Western Allies at the end of the war. The Americans focused their reverse engineering efforts via Operation Paperclip, which led to the development of the PGM-11 Redstone rocket.<sup>[40]</sup> The Soviets used captured German engineers to reproduce technical documents and plans and worked from captured hardware to make their clone of the rocket, the R-1. Thus began the postwar Soviet rocket program, which led to the R-7 and the beginning of the space race.
- K-13/R-3S missile (NATO reporting name AA-2 Atoll), a Soviet reverse-engineered copy of the AIM-9 Sidewinder, was made possible after a Taiwanese (ROCAF) AIM-9B hit a Chinese PLA MiG-17 without exploding in September 1958.<sup>[41]</sup> The missile became lodged within the airframe, and the pilot returned to base with what Soviet scientists would describe as a university course in missile development.
- Toophan missile: In May 1975, negotiations between Iran and Hughes Missile Systems on co-production of the BGM-71 TOW and Maverick missiles stalled over disagreements in the pricing structure, the subsequent 1979 revolution ending all plans for such co-production. Iran was later successful in reverse-engineering the missile and now produces its own copy, the Toophan.
- China has reverse engineered many examples of Western and Russian hardware, from fighter aircraft to missiles and HMMWV cars, such as the MiG-15,17,19,21 (which became the J-2,5,6,7) and the Su-33 (which became the J-15).<sup>[42]</sup>
- During the Second World War, Polish and British cryptographers studied captured German "Enigma" message encryption machines for weaknesses. Their operation was then simulated on electromechanical devices, "bombes", which tried all the possible scrambler settings of the "Enigma" machines that helped the breaking of coded messages that had been sent by the Germans.
- Also during the Second World War, British scientists analyzed and defeated a series of increasingly-sophisticated radio navigation systems used by the Luftwaffe to perform guided bombing missions at night. The British countermeasures to the system were so effective that in some cases, German aircraft were led by signals to land at RAF bases since they believed that they had returned to German territory.

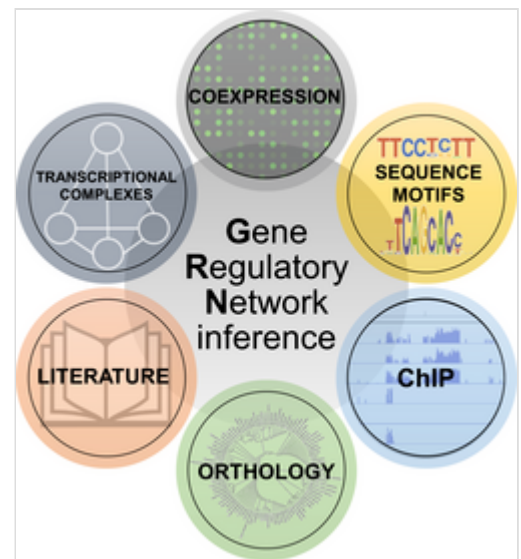
## Gene networks

Reverse engineering concepts have been applied to biology as well, specifically to the task of understanding the structure and function of gene regulatory networks. They regulate almost every aspect of biological behavior and allow cells to carry out physiological processes and responses to perturbations. Understanding the structure and the dynamic behavior of gene networks is therefore one of the paramount challenges of systems biology, with immediate practical repercussions in several applications that are beyond basic research.<sup>[43]</sup>



There are several methods for reverse engineering gene regulatory networks by using molecular biology and data science methods. They have been generally divided into six classes:<sup>[44]</sup>

- Coexpression methods are based on the notion that if two genes exhibit a similar expression profile, they may be related although no causation can be simply inferred from coexpression.
- Sequence motif methods analyze gene promoters to find specific transcription factor binding domains. If a transcription factor is predicted to bind a promoter of a specific gene, a regulatory connection can be hypothesized.
- Chromatin ImmunoPrecipitation (ChIP) methods investigate the genome-wide profile of DNA binding of chosen transcription factors to infer their downstream gene networks.
- Orthology methods transfer gene network knowledge from one species to another.
- Literature methods implement text mining and manual research to identify putative or experimentally-proven gene network connections.
- Transcriptional complexes methods leverage information on protein-protein interactions between transcription factors, thus extending the concept of gene networks to include transcriptional regulatory complexes.



The six classes of gene network inference methods, according to<sup>[44]</sup>

Often, gene network reliability is tested by genetic perturbation experiments followed by dynamic modelling, based on the principle that removing one network node has predictable effects on the functioning of the remaining nodes of the network.<sup>[45]</sup> Applications of the reverse engineering of gene networks range from understanding mechanisms of plant physiology<sup>[46]</sup> to the highlighting of new targets for anticancer therapy.<sup>[47]</sup>

## Overlap with patent law

Reverse engineering applies primarily to gaining understanding of a process or artifact in which the manner of its construction, use, or internal processes has not been made clear by its creator.

Patented items do not of themselves have to be reverse-engineered to be studied, for the essence of a patent is that inventors provide a detailed public disclosure themselves, and in return receive legal protection of the invention that is involved. However, an item produced under one or more patents could also include other technology that is not patented and not disclosed. Indeed, one common motivation of reverse engineering is to determine whether a competitor's product contains patent infringement or copyright infringement.

# Legality

---

## United States

In the United States, even if an artifact or process is protected by trade secrets, reverse-engineering the artifact or process is often lawful if it has been legitimately obtained.<sup>[48]</sup>

Legal analysis has shown that reverse engineering may qualify as fair use when the intent is to study how a system functions, or conduct security research, assuming the copyright property is not redistributed<sup>[49]</sup> Court rulings such as *\*Sega Enterprises Ltd. v. Accolade, Inc.\** have further clarified that reverse engineering to gain access to unprotected functional elements of software can be lawful under U.S. copyright doctrine.<sup>[50]</sup>

Reverse engineering of computer software often falls under both contract law as a breach of contract as well as any other relevant laws. That is because most end-user license agreements specifically prohibit it, and US courts have ruled that if such terms are present, they override the copyright law that expressly permits it (see *Bowers v. Baystate Technologies*<sup>[51][52]</sup>). According to Section 103(f) of the Digital Millennium Copyright Act (17 U.S.C. § 1201 (f) (<https://www.law.cornell.edu/uscode/text/17/1201->)), a person in legal possession of a program may reverse-engineer and circumvent its protection if that is necessary to achieve "interoperability", a term that broadly covers other devices and programs that can interact with it, make use of it, and to use and transfer data to and from it in useful ways. A limited exemption exists that allows the knowledge thus gained to be shared and used for interoperability purposes.<sup>[a]</sup>

## European Union

EU Directive 2009/24 on the legal protection of computer programs, which superseded an earlier (1991) directive,<sup>[53]</sup> governs reverse engineering in countries of the European Union.<sup>[54][b]</sup>

## See also

---

- AI-assisted reverse engineering – Branch of computer science
- Antikythera mechanism – Ancient Greek analogue astronomical computer
- Backward induction – Process of reasoning backwards in sequence
- Benchmarking – Comparing business metrics in an industry
- Bus analyzer
- Chonda – Type of Honda engine
- Clone (computing) – Duplicate of a system
- Clean room design – Reverse-engineering without infringing copyright
- Coordinate-measuring machine (CMM) – Device for measuring the geometry of objects
- Code morphing – Approach used in obfuscating software
- Connectix Virtual Game Station
- Counterfeiting – Making a copy or imitation which is represented as the original

- Cryptanalysis – Study of analyzing information systems in order to discover their hidden aspects
- Decompile – Program translating executable to source code
- Deformulation – Set of analytical procedures in chemistry
- Digital Millennium Copyright Act (DMCA) – 1998 United States copyright law
- Disassembler – Computer program to translate machine language into assembly language
- Dongle – Small piece of computer hardware
- Forensic engineering – Investigation of failures associated with legal intervention
- Industrial CT scanning – Computer-aided tomographic process
- Interactive Disassembler – Software reverse engineering tool
- Knowledge Discovery Metamodel – Specification of Object Management Group
- Laser scanner – Scanning of an object or environment to collect data on its shape
- List of production topics
- Listeroid Engines – British manufacturer of agricultural machinery
- Logic analyzer – Electronic test instrument that measures multiple signals from a circuit
- Paycheck (film) – 2003 American science fiction action film by John Woo
- Repurposing – Using object intended for one purpose in alternative way
- Retrodiction (Thought experimentation) – Hypothetical situation
- Reverse architecture – Concept in systems analysis
- Round-trip engineering
- Sega v. Accolade – 1992 American court case
- Software archaeology – Study of legacy software implementations
- Software cracking – Modification of software, often to use it for free
- Value engineering – Engineering analysis that maximizes function-to-cost ratio

## Notes

---

a. The section states:

(f) Reverse Engineering.—

(1) Notwithstanding the provisions of subsection (a)(1)(A), a person who has lawfully obtained the right to use a copy of a computer program may circumvent a technological measure that effectively controls access to a particular portion of that program for the sole purpose of identifying and analyzing those elements of the program that are necessary to achieve interoperability of an independently created computer program with other programs, and that have not previously been readily available to the person engaging in the circumvention, to the extent any such acts of identification and analysis do not constitute infringement under this title.

(2) Notwithstanding the provisions of subsections (a)(2) and (b), a person may develop and employ technological means to circumvent a technological measure, or to circumvent protection afforded by a technological measure, in order to enable the identification and analysis under paragraph (1), or for the purpose of enabling interoperability of an independently created computer program with other programs, if such means are necessary to achieve such interoperability, to the extent that doing so does not constitute infringement under this title.

(3) The information acquired through the acts permitted under paragraph (1), and the means permitted under paragraph (2), may be made available to others if the person referred to in paragraph (1) or (2), as the case may be, provides such information or means solely for the purpose of enabling interoperability of an independently created computer program with other programs, and to the extent that doing so does not constitute infringement under this

title or violate applicable law other than this section.

(4) For purposes of this subsection, the term 「interoperability」 means the ability of computer programs to exchange information, and of such programs mutually to use the information which has been exchanged.

b. The directive states:

The unauthorised reproduction, translation, adaptation or transformation of the form of the code in which a copy of a computer program has been made available constitutes an infringement of the exclusive rights of the author. Nevertheless, circumstances may exist when such a reproduction of the code and translation of its form are indispensable to obtain the necessary information to achieve the interoperability of an independently created program with other programs. It has therefore to be considered that, in these limited circumstances only, performance of the acts of reproduction and translation by or on behalf of a person having a right to use a copy of the program is legitimate and compatible with fair practice and must therefore be deemed not to require the authorisation of the rightholder. An objective of this exception is to make it possible to connect all components of a computer system, including those of different manufacturers, so that they can work together. Such an exception to the author's exclusive rights may not be used in a way which prejudices the legitimate interests of the rightholder or which conflicts with a normal exploitation of the program.

## References

---

1. "What is Reverse-engineering? How Does It Work" (<https://www.techtarget.com/searchsoftwarequality/definition/reverse-engineering>). *SearchSoftwareQuality*. Retrieved 27 July 2022.
2. "Reverse Engineering" (<https://ethics.csc.ncsu.edu/intellectual/reverse/study.php>). *ethics.csc.ncsu.edu*. Retrieved 27 July 2022.
3. Garcia, Jorge (December 2015). "Un-building blocks: a model of reverse engineering and applicable heuristics" (<https://core.ac.uk/download/pdf/36740023.pdf>) (PDF). *Core.ac.uk*. Retrieved 4 June 2023.
4. **Cite error: The named reference :1 was invoked but never defined (see the help page).**
5. Thayer, Ken. "How Does Reverse Engineering Work?" (<https://insights.globalspec.com/article/7367/how-does-reverse-engineering-work>). *globalspec*. IEEE Global Spec. Retrieved 26 February 2018.
6. Villaverde, Alejandro F.; Banga, Julio R. (6 February 2014). "Reverse engineering and identification in systems biology: strategies, perspectives and challenges" (<https://www.ncbi.nlm.nih.gov/pmc/articles/PMC3869153>). *Journal of the Royal Society Interface*. **11** (91) 20130505. doi:10.1098/rsif.2013.0505 (<https://doi.org/10.1098%2Frsif.2013.0505>). PMC 3869153 (<https://www.ncbi.nlm.nih.gov/pmc/articles/PMC3869153>). PMID 24307566 (<https://pubmed.ncbi.nlm.nih.gov/24307566>).
7. Chikofsky, E.J. & Cross, J.H. II (1990). "Reverse Engineering and Design Recovery: A Taxonomy". *IEEE Software*. **7** (1): 13–17. Bibcode:1990ISoft...7a..13C (<https://ui.adsabs.harvard.edu/abs/1990ISoft...7a..13C>). doi:10.1109/52.43044 (<https://doi.org/10.1109%2F52.43044>). S2CID 16266661 (<https://api.semanticscholar.org/CorpusID:16266661>).

8. A Survey of Reverse Engineering and Program Comprehension. Michael L. Nelson, April 19, 1996, ODU CS 551 – Software Engineering Survey. [arXiv:cs/0503068v1](https://arxiv.org/abs/cs/0503068v1) (<https://arxiv.org/abs/cs/0503068v1>)
9. Vinesh Raja; Kiran J. Fernandes (2007). *Reverse Engineering: An Industrial Perspective*. Springer Science & Business Media. p. 3. ISBN 978-1-84628-856-2.
10. Jonathan Band; Masanobu Katoh (2011). *Interfaces on Trial 2.0*. MIT Press. p. 136. ISBN 978-0-262-29446-1.
11. Chikofsky, E. J.; Cross, J. H. (January 1990). "Reverse engineering and design recovery: A taxonomy" (<https://web.archive.org/web/20180417124021/http://win.ua.ac.be/~lore/Research/Chikofsky1990-Taxonomy.pdf>) (PDF). *IEEE Software*. **7** (1): 13–17. Bibcode:1990ISoft...7a..13C (<https://ui.adsabs.harvard.edu/abs/1990ISoft...7a..13C>). doi:10.1109/52.43044 (<https://doi.org/10.1109%2F52.43044>). S2CID 16266661 (<https://api.semanticscholar.org/CorpusID:16266661>). Archived from the original (<http://win.ua.ac.be/~lore/Research/Chikofsky1990-Taxonomy.pdf>) (PDF) on 17 April 2018. Retrieved 2 July 2012.
12. Eilam, Eldad (2005). *Reversing: secrets of reverse engineering*. John Wiley & Sons. ISBN 978-0-7645-7481-8.
13. "ResHax" (<https://reshax.com/>). *ResHax*. 12 November 2025. Retrieved 19 November 2025.
14. Internet Engineering Task Force RFC 2828 Internet Security Glossary
15. Karwowski, Waldemar; Trzcielinski, Stefan; Mrugalsk, Beata; DiNicolantonio, Massimo; Rossi, Emilio (2018). *Advances in Manufacturing, Production Management and Process Control*. pp. 287–288.
16. Varady, T; Martin, R; Cox, J (1997). "Reverse engineering of geometric models—an introduction". *Computer-Aided Design*. **29** (4): 255–268. Bibcode:1997CAD....29..255V (<https://ui.adsabs.harvard.edu/abs/1997CAD....29..255V>). doi:10.1016/S0010-4485(96)00054-1 (<https://doi.org/10.1016%2FS0010-4485%2896%2900054-1>).
17. "Reverse Engineering" (<http://www.physicaldigital.com/services/reverse-engineering/>). *Physical Digital*.
18. Patel, Suresh (25 August 2022). "Reverse Engineering a Printed Circuit Board" (<https://www.electronicdesign.com/markets/automation/article/21249478/mer-mar-electronics-reverse-engineering-a-printed-circuit-board>). *Electronic Design*. Retrieved 1 February 2024.
19. Patel, Suresh (25 August 2022). "Reverse Engineering a Printed Circuit Board" (<https://www.electronicdesign.com/markets/automation/article/21249478/mer-mar-electronics-reverse-engineering-a-printed-circuit-board>). *Electronic Design*. Retrieved 1 February 2024.
20. Warden, R. (1992). *Software Reuse and Reverse Engineering in Practice*. London, England: Chapman & Hall. pp. 283–305.
21. "Working Conference on Reverse Engineering (WCRE)" (<https://web.archive.org/web/20170314150255/http://dblp.uni-trier.de/db/conf/wcre/index.html>). *uni-trier.de*. Computer Science bibliography. Archived from the original (<http://dblp.uni-trier.de/db/conf/wcre/index.html>) on 14 March 2017. Retrieved 22 February 2018.
22. Shahbaz, Muzammil (2012). *Reverse Engineering and Testing of Black-Box Software Components: by Grammatical Inference techniques*. LAP LAMBERT Academic Publishing. ISBN 978-3-659-14073-0.
23. Chuvakin, Anton; Cyrus Peikari (January 2004). *Security Warrior* (<https://web.archive.org/web/20060522102255/http://www.oreilly.com/catalog/swarrior/reviews.html>) (1st ed.). O'Reilly. Archived from the original (<http://www.oreilly.com/catalog/swarrior/reviews.html>) on 22 May 2006. Retrieved 25 May 2006.

24. Samuelson, Pamela & Scotchmer, Suzanne (2002). "The Law and Economics of Reverse Engineering" (<https://web.archive.org/web/20100715135359/http://yalelawjournal.org/the-yale-law-journal/content-pages/the-law-and-economics-of-reverse-engineering/>). *Yale Law Journal*. **111** (7): 1575–1663. doi:10.2307/797533 (<https://doi.org/10.2307%2F797533>). JSTOR 797533 (<https://www.jstor.org/stable/797533>). Archived from the original (<http://www.yalelawjournal.org/the-yale-law-journal/content-pages/the-law-and-economics-of-reverse-engineering/>) on 15 July 2010. Retrieved 31 October 2011.
25. "Samba: An Introduction" (<https://www.samba.org/samba/docs/SambaIntro.html>). 27 November 2001. Retrieved 7 May 2009.
26. W. Cui, J. Kannan, and H. J. Wang. Discoverer: Automatic protocol reverse engineering from network traces. In Proceedings of 16th USENIX Security Symposium on USENIX Security Symposium, pp. 1–14.
27. W. Cui, M. Peinado, K. Chen, H. J. Wang, and L. Irún-Briz. Tupni: Automatic reverse engineering of input formats. In Proceedings of the 15th ACM Conference on Computer and Communications Security, pp. 391–402. ACM, Oct 2008.
28. P. M. Comparetti, G. Wondracek, C. Kruegel, and E. Kirda. Prospex: Protocol specification extraction. In Proceedings of the 2009 30th IEEE Symposium on Security and Privacy, pp. 110–125, Washington, 2009. IEEE Computer Society.
29. Gold, E (1978). "Complexity of automaton identification from given data" (<https://doi.org/10.1016%2FS0019-9958%2878%2990562-4>). *Information and Control*. **37** (3): 302–320. doi:10.1016/S0019-9958(78)90562-4 (<https://doi.org/10.1016%2FS0019-9958%2878%2990562-4>).
30. D. Angluin (1987). "Learning regular sets from queries and counterexamples" (<https://doi.org/10.1016%2F0890-5401%2887%2990052-6>). *Information and Computation*. **75** (2): 87–106. doi:10.1016/0890-5401(87)90052-6 (<https://doi.org/10.1016%2F0890-5401%2887%2990052-6>).
31. C.Y. Cho, D. Babic, R. Shin, and D. Song. Inference and Analysis of Formal Models of Botnet Command and Control Protocols (<https://web.archive.org/web/20101028102651/http://www.domagoj-babic.com/index.php/Pubs/CCS10botnets>), 2010 ACM Conference on Computer and Communications Security.
32. Polyglot: automatic extraction of protocol message format using dynamic binary analysis ([http://bitblaze.cs.berkeley.edu/papers/polyglot\\_ccs07\\_av.pdf](http://bitblaze.cs.berkeley.edu/papers/polyglot_ccs07_av.pdf)). J. Caballero, H. Yin, Z. Liang, and D. Song. Proceedings of the 14th ACM conference on Computer and communications security, pp. 317–329.
33. Wolfgang Rankl, Wolfgang Effing, Smart Card Handbook (2004)
34. T. Welz: Smart cards as methods for payment ([http://www.crypto.rub.de/its\\_seminar\\_ws0708.html](http://www.crypto.rub.de/its_seminar_ws0708.html)) (2008), Seminar ITS-Security Ruhr-Universität Bochum
35. David C. Musker: Protecting & Exploiting Intellectual Property in Electronics (<http://www.jenkins.eu/articles-general/reverse-engineering.asp>) Archived (<https://web.archive.org/web/20110709001411/http://www.jenkins.eu/articles-general/reverse-engineering.asp>) 2011-07-09 at the Wayback Machine, IBC Conferences, 10 June 1998
36. Francillon, René J. (1988) [1979]. *McDonnell Douglas Aircraft since 1920*. Vol. 1 (2nd ed.). Annapolis, Maryland: Naval Institute Press. pp. 265–268. ISBN 0-87021-428-4.
37. Westell, Freeman (November 1999). "Big Iron, big engines & bigger headaches: Building the first experimental strategic bombers". *Airpower*. **29** (6): 18, 49–50.
38. Angelucci, Enzo; Matricardi, Paolo (1978). *-World War II Airplanes: Volume 2*. Chicago: Rand McNally & Company. p. 127. ISBN 0-528-88171-X.
39. Yeam Gordon and Vladimir Rigmant, Tupolev Tu-4: Soviet Superfortress (Hinckley, U.K.: Midland, 2002).
40. "Redstone rocket" (<http://www.centennialofflight.net/essay/Dictionary/REDSTONE/DI149.htm>). centennialofflight.net. Retrieved 27 April 2010.

41. "The Chinese Air Force: Evolving Concepts, Roles, and Capabilities", Center for the Study of Chinese Military Affairs (U.S), by National Defense University Press, p. 277
42. Chandrashekar, S., R. Nagappa, L. Sundaresan, and N. Ramani. 2011. Technology & Innovation in China: A Case Study of Single Crystal Superalloy Development for Aircraft Turbine Blades, R4–11. ISSSP National Institute of Advanced Studies, Bangalore.  
<http://issp.in/wp-content/uploads/2013/01/Technology-and-Innovation-in-China-A-case-Study-of-Single-Crystal4.pdf>; and Dillon Zhou, "China J-15 Fighter Jet: Chinese Officials Defend New Fighter As Chinese Original, but Questions Remain," Mic, December 16, 2012, <https://www.mic.com/articles/20270/china-j-15-fighter-jet-chinese-officials-defend-new-fighter-as-chinese-original-but-questions-remain>
43. Giorgi, Federico M. (2020). "Gene network reverse engineering: The Next Generation". *Biochimica et Biophysica Acta (BBA) - Gene Regulatory Mechanisms*. **1863** (6) 194523. doi:10.1016/j.bbagr.2020.194523 (<https://doi.org/10.1016%2Fj.bbagr.2020.194523>). hdl:11585/753853 (<https://hdl.handle.net/11585%2F753853>). ISSN 1874-9399 (<https://search.worldcat.org/issn/1874-9399>). PMID 32145356 (<https://pubmed.ncbi.nlm.nih.gov/32145356>). S2CID 212629142 (<https://api.semanticscholar.org/CorpusID:212629142>).
44. Mercatelli, Daniele; Scalambra, Laura; Triboli, Luca; Ray, Forest; Giorgi, Federico M. (2020). "Gene regulatory network inference resources: A practical overview". *Biochimica et Biophysica Acta (BBA) - Gene Regulatory Mechanisms*. **1863** (6) 194430. doi:10.1016/j.bbagr.2019.194430 (<https://doi.org/10.1016%2Fj.bbagr.2019.194430>). ISSN 1874-9399 (<https://search.worldcat.org/issn/1874-9399>). PMID 31678629 (<https://pubmed.ncbi.nlm.nih.gov/31678629>). S2CID 207895066 (<https://api.semanticscholar.org/CorpusID:207895066>).
45. Tegner, J.; Yeung, M. K. S.; Hasty, J.; Collins, J. J. (2003). "Reverse engineering gene networks: Integrating genetic perturbations with dynamical modeling" (<https://www.ncbi.nlm.nih.gov/pmc/articles/PMC156306>). *Proceedings of the National Academy of Sciences*. **100** (10): 5944–5949. Bibcode:2003PNAS..100.5944T (<https://ui.adsabs.harvard.edu/abs/2003PNAS..100.5944T>). doi:10.1073/pnas.0933416100 (<https://doi.org/10.1073%2Fpnas.0933416100>). ISSN 0027-8424 (<https://search.worldcat.org/issn/0027-8424>). PMC 156306 (<https://www.ncbi.nlm.nih.gov/pmc/articles/PMC156306>). PMID 12730377 (<https://pubmed.ncbi.nlm.nih.gov/12730377>).
46. Friedel, Svetlana; Usadel, Björn; von Wirén, Nicolaus; Sreenivasulu, Nese (2012). "Reverse Engineering: A Key Component of Systems Biology to Unravel Global Abiotic Stress Cross-Talk" (<https://www.ncbi.nlm.nih.gov/pmc/articles/PMC3533172>). *Frontiers in Plant Science*. **3**: 294. Bibcode:2012FrPS....3..294F (<https://ui.adsabs.harvard.edu/abs/2012FrPS....3..294F>). doi:10.3389/fpls.2012.00294 (<https://doi.org/10.3389%2Ffpls.2012.00294>). ISSN 1664-462X (<https://search.worldcat.org/issn/1664-462X>). PMC 3533172 (<https://www.ncbi.nlm.nih.gov/pmc/articles/PMC3533172>). PMID 23293646 (<https://pubmed.ncbi.nlm.nih.gov/23293646>).
47. Lefebvre, Celine; Rieckhof, Gabrielle; Califano, Andrea (2012). "Reverse-engineering human regulatory networks" (<https://www.ncbi.nlm.nih.gov/pmc/articles/PMC4128340>). *Wiley Interdisciplinary Reviews: Systems Biology and Medicine*. **4** (4): 311–325. doi:10.1002/wsbm.1159 (<https://doi.org/10.1002%2Fwsbm.1159>). ISSN 1939-5094 (<https://search.worldcat.org/issn/1939-5094>). PMC 4128340 (<https://www.ncbi.nlm.nih.gov/pmc/articles/PMC4128340>). PMID 22246697 (<https://pubmed.ncbi.nlm.nih.gov/22246697>).
48. "Trade Secrets 101", Feature Article, March 2011 (<https://www.asme.org/career-education/articles/management-professional-practice/trade-secrets-101>). ASME. Retrieved on 2013-10-31.
49. Samuelson, P. (2003). "Reverse Engineering Under Copyright Law." *Communications of the ACM*, 46(8), 15–20.
50. *Sega Enterprises Ltd. v. Accolade, Inc.*, 977 F.2d 1510 (United States Court of Appeals for the Ninth Circuit 1992).



51. Baystate v. Bowers Discussion (<http://www.utsystem.edu/ogc/intellectualproperty/baystatevbowersdiscussion.htm>). Utsystem.edu. Retrieved on 2011-05-29.
52. Gross, Grant. (2003-06-26) Contract case could hurt reverse engineering | Developer World (<http://www.infoworld.com/d/developer-world/contract-case-could-hurt-reverse-engineering-337>). InfoWorld. Retrieved on 2011-05-29.
53. Council Directive 91/250/EEC of 14 May 1991 on the legal protection of computer programs (<https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX:01991L0250-19931119>)
54. Directive 2009/24/EC of the European Parliament and of the Council of 23 April 2009 on the legal protection of computer programs (Codified version) (<https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX:32009L0024>)

## Sources

---

- Eilam, Eldad (2005). *Reversing: Secrets of Reverse Engineering*. Wiley Publishing. p. 595. ISBN 978-0-7645-7481-8.
- Elvidge, Julia, "Using Reverse Engineering to Discover Patent Infringement," Chipworks, Sept. 2010. Online: <http://www.photonics.com/Article.aspx?AID=44063>
- Cipresso, Teodoro (2009). "Software Reverse Engineering Education" (<http://www.reversingproject.info>). *SJSU Master's Thesis*. Retrieved 22 August 2009.
- Hausi A. Müller and Holger M. Kienle, "A Small Primer on Software Reverse Engineering," Technical Report, University of Victoria, 17 pages, March 2009. Online: <http://holgerkienle.wikispaces.com/file/view/MK-UVic-09.pdf>
- Heines, Henry, "Determining Infringement by X-Ray Diffraction," *Chemical Engineering Process*, Jan. 1999 (example of reverse engineering used to detect IP infringement)
- Huang, Andrew (2003). *Hacking the Xbox: An Introduction to Reverse Engineering*. No Starch Press. ISBN 978-1-59327-029-2.
- James, Dick (19 January 2006). "Reverse Engineering Delivers Product Knowledge; Aids Technology Spread" (<http://electronicdesign.com/Articles/Index.cfm?AD=1&ArticleID=11966>). *Electronic Design*. Penton Media, Inc. Retrieved 3 February 2009.
- Messler, Robert (2013). *Reverse Engineering: Mechanisms, Structures, Systems & Materials*. McGraw Hill. ISBN 978-0-07-182516-0. (introduction to hardware teardowns, including methodology, goals)
- Raja, Vinesh; Fernandes, Kiran J. (2008). *Reverse Engineering – An Industrial Perspective*. Springer. p. 242. ISBN 978-1-84628-855-5.
- Samuelson, Pamela and Scotchmer, Suzanne, "The Law and Economics of Reverse Engineering," 111 Yale L.J. 1575 (2002). Online: <http://people.ischool.berkeley.edu/~pam/papers/l&e%20reveng3.pdf>
- Schulman, Andrew; Brown, Ralf D.; Maxey, David; Michels, Raymond J.; Kyle, Jim (1994) [November 1993]. *Undocumented DOS: A programmer's guide to reserved MS-DOS functions and data structures - expanded to include MS-DOS 6, Novell DOS and Windows 3.1* ([https://archive.org/details/undocumenteddos00andr\\_0/page/229](https://archive.org/details/undocumenteddos00andr_0/page/229)) (2 ed.). Reading, Massachusetts: Addison Wesley. pp. 229–241 ([https://archive.org/details/undocumenteddos00andr\\_0/page/229](https://archive.org/details/undocumenteddos00andr_0/page/229)). ISBN 0-201-63287-X. (xviii+856+vi pages, 3.5"-floppy) Errata: [1] (<https://web.archive.org/web/20190417215556/http://www.cs.cmu.edu/afs/cs/user/ralf/pub/books/UndocumentedDOS/errata.ud2>)[2] ([https://web.archive.org/web/20190417212906/https://www.pcjs.org/pubs/pc/programming/Undocumented\\_DOS/#errata-2nd-edition](https://web.archive.org/web/20190417212906/https://www.pcjs.org/pubs/pc/programming/Undocumented_DOS/#errata-2nd-edition)) (NB. On general methodology of reverse engineering, applied to mass-market software: a program for exploring DOS, disassembling DOS.)



- Schulman, Andrew; et al. (1992). *Undocumented Windows: A Programmer's Guide to Reserved Microsoft Windows API Functions* (<https://archive.org/details/undocumentedwind00schu>). Addison Wesley. ISBN 978-0-201-60834-2. (pp. 59–188 on general methodology of reverse engineering, applied to mass-market software: examining Windows executables, disassembling Windows, tools for exploring Windows)
- Schulman, Andrew, "Hiding in Plain Sight: Using Reverse Engineering to Uncover Software Patent Infringement," *Intellectual Property Today*, Nov. 2010. Online: <http://www.iptoday.com/issues/2010/11/hiding-in-plain-sight-using-reverse-engineering-to-uncover-software-patent-infringement.asp>
- Schulman, Andrew, "Open to Inspection: Using Reverse Engineering to Uncover Software Prior Art," *New Matter* (Calif. State Bar IP Section), Summer 2011 (Part 1); Fall 2011 (Part 2). Online: <http://www.SoftwareLitigationConsulting.com>
- Thumm, Mike (2007). "Talking Tactics" ([https://ewh.ieee.org/r5/denver/sscs/References/2007\\_09\\_Torrance.pdf](https://ewh.ieee.org/r5/denver/sscs/References/2007_09_Torrance.pdf)) (PDF). *IEEE 2007 Custom Integrated Circuits Conference (CICC)*. IEEE, Inc. Retrieved 3 February 2009.

---

Retrieved from "[https://en.wikipedia.org/w/index.php?title=Reverse\\_engineering&oldid=1352150450](https://en.wikipedia.org/w/index.php?title=Reverse_engineering&oldid=1352150450)"